



AN 977: Nios® V Processor Custom Instruction



Online Version



Send Feedback

ID: **773194**

Version: **2023.04.14**

Contents

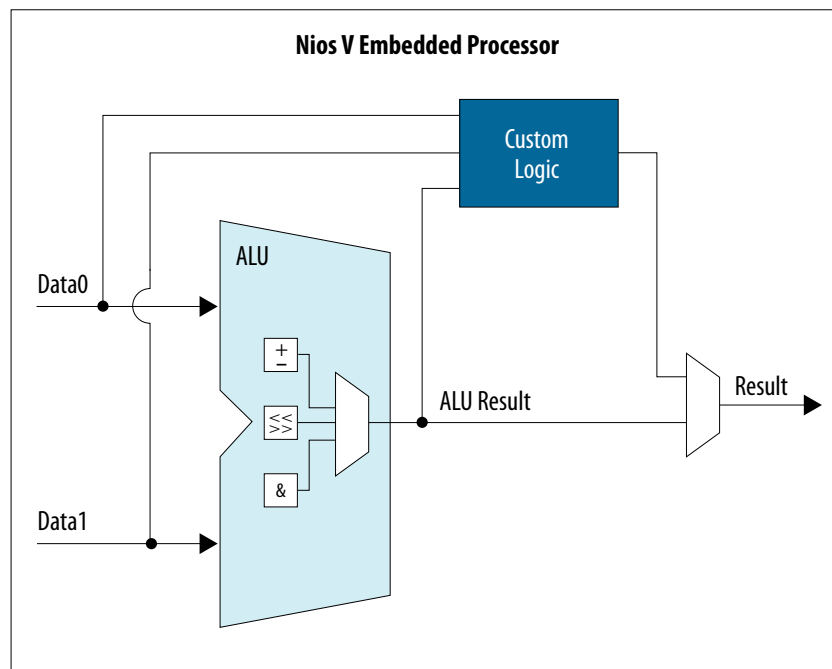
1. Nios® V Processor Custom Instruction Overview.....	3
1.1. Implementing Custom Instruction.....	4
1.2. Implementing Custom Instruction Logic Block.....	5
1.3. Implementing Custom Instruction Software.....	6
2. Custom Instruction Hardware Interface.....	7
2.1. Custom Instruction Types.....	7
2.2. Multicycle Custom Instructions.....	7
2.2.1. Multicycle Custom Instruction Ports.....	7
2.2.2. Multicycle Custom Instruction Timing.....	8
2.2.3. Implementing Combinational Custom Instruction.....	9
2.3. Extended Custom Instruction.....	9
2.3.1. Extended Custom Instruction Timing.....	11
2.4. External Interface Custom Instructions.....	11
3. Custom Instruction Software Interface.....	13
3.1. Custom Instruction Software C Macro.....	13
3.2. Custom Instruction Assembly Language Interface.....	14
3.2.1. Custom Instruction Assembly Language Syntax.....	14
3.3. Custom Instruction Word Format.....	14
3.3.1. Selector and Extension Indexes.....	15
3.3.2. ALU Switch.....	16
3.3.3. User-defined Immediate.....	18
4. Designing with Intel Quartus Prime and Platform Designer.....	19
4.1. Interface Signals.....	19
4.1.1. alu_result port.....	19
4.1.2. ctrl port.....	19
4.2. Custom Instruction Manager.....	20
4.2.1. Hardware Interfaces Table.....	20
4.2.2. Software C-Macro Table.....	21
4.3. Custom Instruction Subordinate.....	23
4.3.1. Writing the Custom Instruction HDL File.....	23
4.3.2. Opening the Component Editor.....	24
4.3.3. Specifying the Custom Instruction Component Type.....	24
4.3.4. Displaying the Custom Instruction Block Symbol.....	24
4.3.5. Adding the Custom Instruction HDL File.....	25
4.3.6. Configuring the Custom Instruction Parameter Type.....	27
4.3.7. Setting Up the Custom Instruction Interfaces.....	28
4.3.8. Saving and Adding the Custom Instruction.....	29
4.3.9. Generating and Compiling the Processor System.....	30
5. Document Revision History for AN 977: Nios V Processor Custom Instruction.....	31

1. Nios® V Processor Custom Instruction Overview

Custom instructions allow you to customize the Nios® V processor to meet the needs of a particular application. You can accelerate time-critical software algorithms by converting them into custom hardware logic blocks. Custom instructions provide an easy way to experiment with hardware-software tradeoffs at any point in the design process because you can easily alter the design of the FPGA-based Nios V processor.

The custom instruction logic connects to the Nios V processor arithmetic logic unit (ALU) as shown in the following figure. The feature is available in the Nios V/g processor variant only.

Figure 1. Custom Instruction Logic Connections to the Nios V Processor ALU



The Nios V processor custom instruction does not support the following operations:

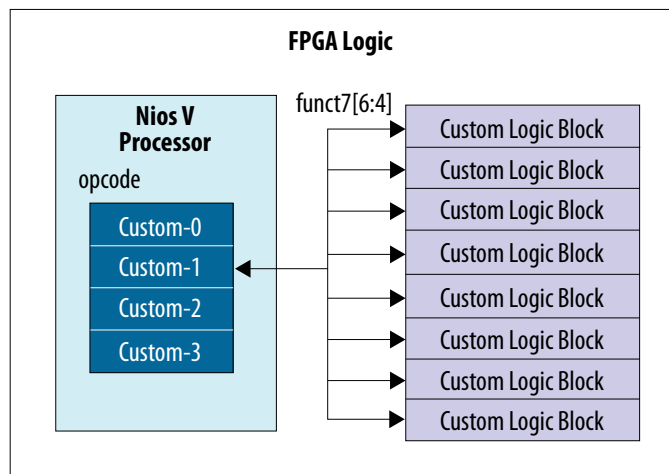
- Branch operations
- Read or write Control and Status Registers (CSRs)
- Modify processor cache

1.1. Implementing Custom Instruction

Nios V processor custom instructions are custom logic blocks adjacent to the arithmetic logic unit (ALU) in the processor's datapath.

Nios V processor custom instructions are based on the RISC-V R-type instruction format, and provides support for four unique opcodes. Leveraging the funct7[6:4] in the custom instruction, each opcode is further associated with eight custom logic blocks. In total, you can have up to 32 custom logic blocks connected to a single Nios V processor. Logic block connections on the Nios V processor are exposed as **Nios V Custom Instruction Manager** interfaces, and are defined using the **Custom Instructions Hardware Interfaces Table** in the Nios V processor IP parameterization.

Figure 2. Custom Logic Blocks



The following steps show how you can implement and use custom instructions with the Nios V processor:

1. When you implement the custom instructions in the Nios V processor system, each custom operation must associate with its unique selector index (opcode and funct7[6:4]).
2. The selector index allows the software to specify the desired operation from among up to 32 custom logic blocks.
3. In the Platform Designer, you can define one or more selector index using the **Custom Instructions Hardware Interfaces Table**. Each defined selector index would be presented as a **Nios V Custom Instruction Manager** interface on the Nios V processor.
4. You can define custom instruction software C macros for software deployment in system.h using the **Custom Instructions Hardware Interfaces Table** in the Nios V IP parameterization. An accompanying GDB debug memonics file for the defined software C macros is generated by Platform Designer for use by the Ashling* RiscFree* IDE for Intel® FPGAs.

Note: You can have up to 128 custom instructions in a single custom logic block using the extended custom instructions. For example, if you instantiate all 32 logic blocks, you can have a maximum of 4096 custom instructions connected to a single Nios V processor.

Related Information

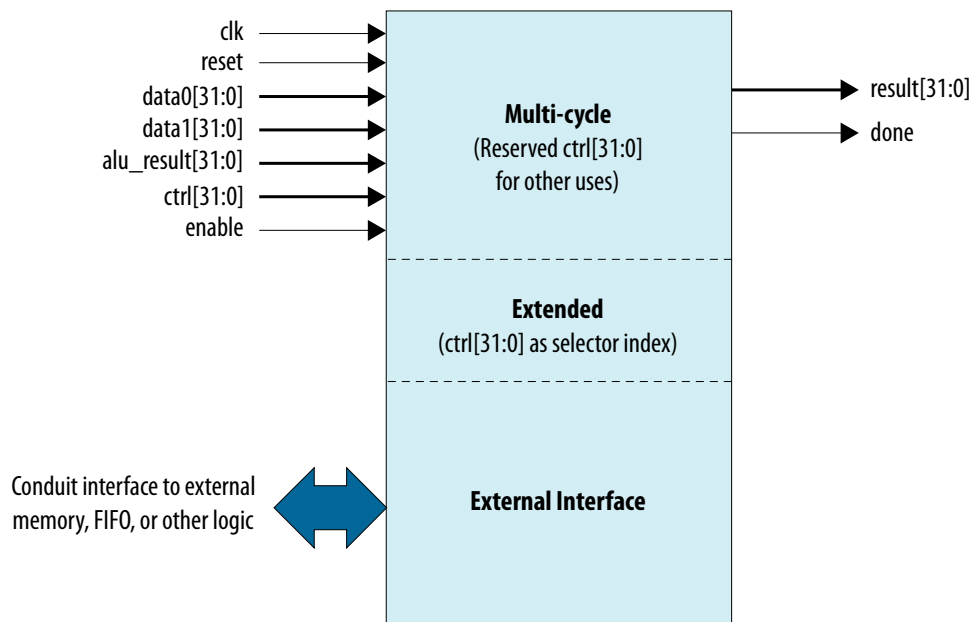
- [Extended Custom Instruction](#) on page 9
For more information about implementing multiple custom instructions per logic block.
- [Custom Instruction Manager](#) on page 20
For more information about exporting selector one or more index into Nios V Processor IP Parameter Editor.
- [Hardware Interfaces Table](#) on page 20
- [Software C-Macro Table](#) on page 21

1.2. Implementing Custom Instruction Logic Block

The Nios V processor supports three types of custom instructions:

- Multicycle
- Extended
- External Interface

Figure 3. Nios V Processor Custom Instruction Logic Block Interface



Custom Instruction logic blocks consists of a Nios V Processor Custom Instruction Subordinate interface. The figure above shows all the ports in the Nios V Custom Instruction Subordinate interface that communicates with the Nios V processor.

- clk
- reset
- data0[31:0]
- data1[31:0]
- alu_result[31:0]
- ctrl[31:0]
- enable
- result[31:0]
- done

All the ports can accommodate all custom instruction types. The figure above also shows a conduit interface to the external logic. The interface to external logic allows you to include a custom interface to system resources outside of the Nios V processor datapath.

Nios V processor custom instructions requires the clk, reset, enable and done ports. The custom instruction logic provides a result based on the inputs provided by the Nios V processor. When the enable port is asserted, the Nios V processor custom instruction logic receives inputs on its data0, data1, and alu_result ports and outputs the result to its result port. When the logic block asserts the done port, the processor considers the result valid.

Related Information

- [Custom Instruction Hardware Interface](#) on page 7
For more information about the custom instructions type.
- [Interface Signals](#) on page 19
For more information about the specialized ports (alu_result and ctrl).

1.3. Implementing Custom Instruction Software

The Nios V processor custom instruction software interface outlines the custom instruction details for the software developer.

For each custom instruction, the Board Support Package Editor generates a macro in the system header file, `system.h` and an accompanying GDB debug mnemonics file, `custom_instruction_debug.xml`. You can use the macro directly in your C or C++ application code. You do not need to program the assembly code to access the custom instructions. A software can also invoke custom instructions in the assembly language. The GDB debug mnemonics file can be used with the Ashling RiscFree IDE for Intel FPGAs to provide debug mnemonics for user defined custom instruction encodings.

Related Information

- [Custom Instruction Software Interface](#) on page 13
More information about the software implementations.

2. Custom Instruction Hardware Interface

2.1. Custom Instruction Types

Table 1. Custom Instruction Types, Applications, and Hardware Ports

Instruction Type	Application	Hardware Ports
Multicycle ⁽¹⁾	Multi-clock cycle custom logic blocks of variable durations	• clk • reset • data0[31:0] • data1[31:0] • alu_result[31:0] • ctrl[31:0](Reserved for other use) • enable • result[31:0] • done
Extended	Custom logic blocks that contain multiple instructions	Standard multicycle custom instruction ports, with ctrl[31:0] as extension index.
External Interface	Custom logic blocks that interface to logic outside of the Nios V processor's datapath.	• Standard multicycle custom instruction ports • User-defined interface to external logic

2.2. Multicycle Custom Instructions

Multicycle (sequential) custom instructions consist of a logic block that requires two or more clock cycles to complete an operation. Multicycle custom instructions complete in variable number of clock cycles. Thus, you need to instantiate the enable and done ports. The enable and done ports participate in a handshaking scheme to determine when the custom instruction execution completes.

2.2.1. Multicycle Custom Instruction Ports

Table 2. Multicycle Custom Instruction Ports

Port Name	Direction	Required?	Description
clk	Input	Yes	System clock.
reset	Input	Yes	Synchronous reset.
<i>continued...</i>			

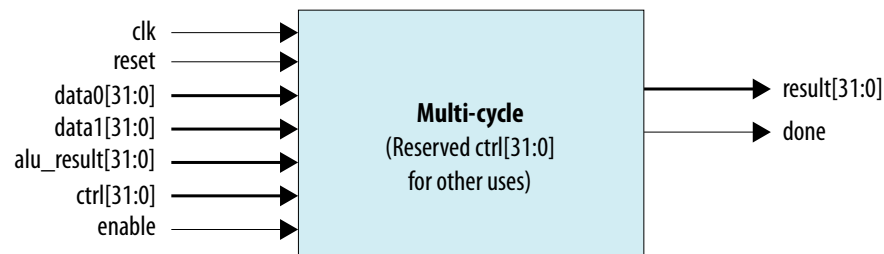
⁽¹⁾ For combinational or single-cycle instructions, you need to implement them as two-cycles instructions.

Port Name	Direction	Required?	Description
data0[31:0]	Input	Optional ⁽²⁾	Input operand to custom instruction logic block (Source Register 1).
data1[31:0]	Input	Optional ⁽²⁾	Input operand to custom instruction logic block (Source Register 2).
alu_result[31:0]	Input	Optional ⁽²⁾	Input operand to custom instruction logic block (ALU Switch result).
ctrl[31:0]	Input	Optional ⁽²⁾	Custom instruction word. Reserved for other use.
enable	Input	Yes	Custom instruction logic block enables.
done	Output	Yes	Indicator to the completion of the custom instruction execution.
result[31:0]	Output	Optional ⁽²⁾	Result of the custom instruction.

The Nios V processor system clock feeds the custom logic block's `clk` port. The Nios V processor system reset feeds the active high `reset` port. The `reset` port asserts only when the whole Nios V processor system resets.

If you assert the `enable` port, the Nios V processor custom instruction logic receives input thru the `data0`, `data1`, and `alu_result` ports, and drives the result to its `result` port. The processor considers the result as valid when the logic block asserts the `done` port. The `ctrl` port delivers the full instruction word to the logic block for other use.

Figure 4. Multicycle Custom Instruction Block Diagram

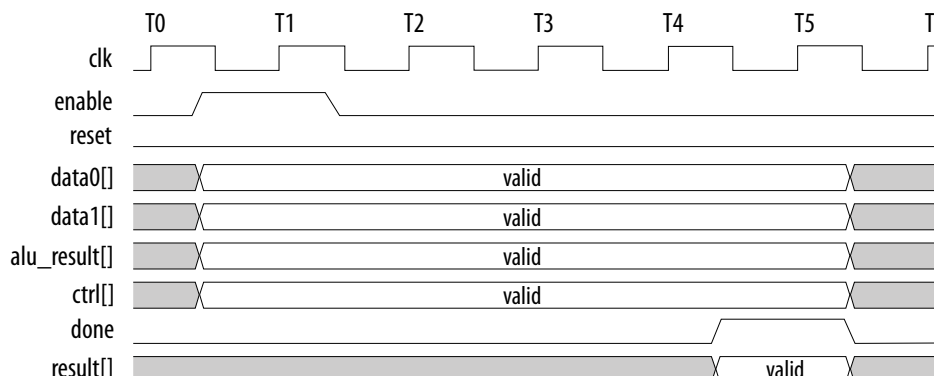


2.2.2. Multicycle Custom Instruction Timing

On the first clock cycle of the custom instruction execution, the processor asserts the active high `enable` port. Simultaneously, the `data0`, `data1`, and `alu_result` ports have valid values and remain valid for the duration of the custom instruction execution. The `enable` signal is asserted for a single clock cycle.

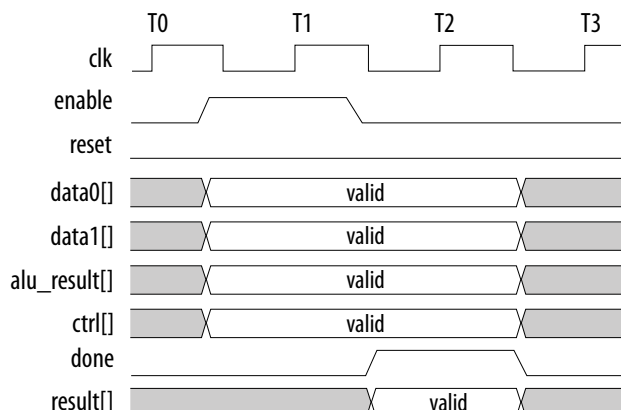
The processor waits until the active high `done` signal is asserted. The processor reads the `result` port on the same clock edge on which `done` is asserted. The custom logic block must present data on the `result` port on the same clock cycle on which it asserts the `done` signal.

⁽²⁾ Use only if the custom instruction requires the port.

Figure 5. Multicycle Custom Instruction Timing Diagram

2.2.3. Implementing Combinational Custom Instruction

A combinational custom instruction is a logic block that completes its logic function in a single clock cycle. To incorporate the combinational instruction into a multicycle custom instruction, combinational custom instruction needs to be implemented in two cycles.

Figure 6. Combinational Custom Instruction Timing Diagram

2.3. Extended Custom Instruction

Extended custom instruction is an extension to multicycle custom instructions. An extended custom instruction allows a single custom logic block to implement several different operations.

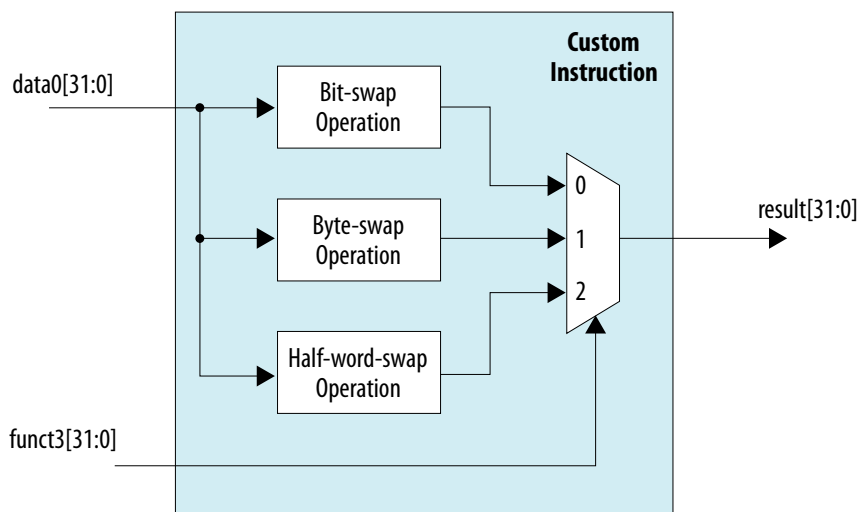
Extended custom instruction components implement multiple indexes, comprised of opcode, funct7, and funct3. These indexes are determined when the custom instruction hardware block is instantiated in the system using Platform Designer.

While the selector indexes (opcode and funct7[6:4]) are required to select the correct logic block, extended custom instructions decode extension indexes (funct7[3:0] and funct3[2:0]) to specify which operation in the logic block to perform. The extension index can be up to seven bits wide, allowing a single custom

logic block to implement as many as 128 different operations. Paired with 32 logic blocks in a single Nios V processor, this enables up to 4096 custom instructions per processor.

The following logic block shows an extended custom instruction with bit-swap, byte-swap, and half-word swap operations. This block performs all swap operations on data received at the `data0` port. The instruction hardware uses the two-bits wide `funct3` port to select the output from a multiplexer, determining which result is presented to the `result` port.

Figure 7. Extended Custom Instruction with Swap Operations



Note: The logic in the figure above is a simple example using a multiplexer on the output. You can implement function selection based on any extension indexes in any way that is appropriate for your application.

To implement an extended custom instruction, apply the `ctrl` port to your custom instruction logic. The total bit width of the `funct7[3:0]` and `funct3[2:0]` port is a function of the number of operations the custom logic block can perform.

Table 3. 32-bit Custom Instruction Word (ctrl)

32-bit Custom Instruction Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
funct7[6:0]								rs2				rs1			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs1		funct3[2:0]				rd				opcode					

Table 4. Example of Selector and Extension Index Application

Selector Index		Extension Index		Supported Custom Instructions (per opcode)
opcode	funct7[6:4]	funct7[3:0]	funct3[2:0]	
custom-0	1 custom logic blocks	<i>Idle</i>	<i>Idle</i>	1 custom instructions
custom-1	8 custom logic blocks	<i>Idle</i>	<i>Idle</i>	8 custom instructions (1 per logic block)
custom-2	8 custom logic blocks	As extension index	<i>Idle</i>	128 custom instructions (16 per logic block)
custom-3	8 custom logic blocks	As extension index	As extension index	1024 custom instructions (128 per logic block)
Total custom instruction in this processor				1161 custom instructions

2.3.1. Extended Custom Instruction Timing

All extended custom instruction port operations are identical to those for the combinational and multicycle custom instructions.

The `ctrl` port timing is the same as that of the `data0` port. The Nios V processor presents the extension index to the `ctrl` port on the same rising edge of the clock at which `enable` is asserted, and the `ctrl` port remains stable during execution of the custom instruction.

Related Information

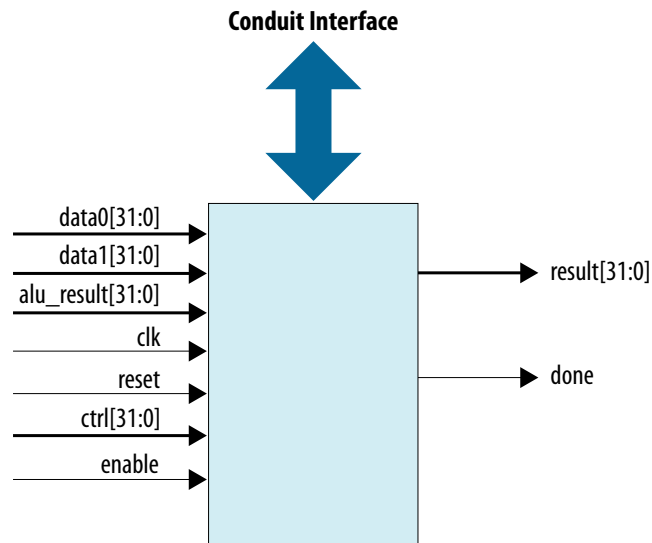
- [Multicycle Custom Instruction Timing](#) on page 8
- [Implementing Combinational Custom Instruction](#) on page 9

2.4. External Interface Custom Instructions

External interface custom instruction is an extension to the multicycle custom instructions. External interface custom instructions allow you to add an interface for communicating with logic that is not in the processor's datapath. You can generate conduits based on the available interconnects. Refer to the *Intel Quartus® Prime Pro Edition User Guide: Platform Designer - Platform Designer Interconnect*.

At system generation, you can export an interface in the Platform Designer by double clicking on the interface to export it allowing external logic to access the signals. By enabling custom instruction logic to access memory external to the processor, external interface custom instructions extend the capabilities of the custom instruction logic.

Figure 8. Custom Instruction with External Interface



Custom instruction logic can perform various tasks such as storing intermediate results or reading memory to control the custom instruction operation. The conduit interface provides a dedicated path for data to flow into or out of the Nios V processor. For example, custom instruction logic with an external interface can feed data directly from the Nios V processor's register file to an external first-in first-out (FIFO) memory buffer.

Related Information

[Intel Quartus Prime Pro Edition User Guide: Platform Designer - Platform Designer Interconnect](#)

For more information about generating conduits based on the available interconnects.

3. Custom Instruction Software Interface

The Nios V processor custom instruction software interface outlines the logic implementation details from the application code.

During the BSP generation, the Board Support Package Editor generate macros that allow access from application code to custom instructions.

3.1. Custom Instruction Software C Macro

The following examples illustrate how the Nios V processor custom instruction software interface fits into the software code.

The example below shows a portion of the `system.h` header file that defines a macro for a custom instruction. The macros defined by the Board Support Package Editor use C integer types only.

Example 1. Software C Macros with assigned funct3 and funct7 fields

```
#define CUSTOM_OPERATION(VAL_1, VAL_2) ({ \
    int output; \
    asm volatile (".insn r 0x0B, 0x1, 0x0, %[out], %[input1], %[input2]" \
: [out] "=r" (output) \
: [input1] "r" (VAL_1), [input2] "r" (VAL_2)); \
    output; \
})
```

Note: The example above assigns funct3 as 0x1 and funct7 as 0x0.

The example below shows an application code that uses the custom instruction and includes the `system.h` file. This example enables the application software to use the custom instruction macro definition. You are required to apply `uint32_t` variable type, along with `inttypes.h` header.

Example 2. Software C Macros that includes `system.h`

```
#include "system.h"
#include <inttypes.h>
int main (void)
{
    uint32_t val_1 = 0x12345678;        //max 32-bit
    uint32_t val_2 = 0x98765432;        //max 32-bit
    uint32_t result;

    result = CUSTOM_OPERATION_1(val_1, val_2);
    return 0;
}
```

3.2. Custom Instruction Assembly Language Interface

You can access the Nios V processor custom instructions in assembly code and C/C++.

3.2.1. Custom Instruction Assembly Language Syntax

Nios V processor custom instructions use a standard assembly language syntax as shown in the following table:

Table 5. Standard Assembly Language

Syntax	Description
<mnemonic>	- Identifies the particular custom instruction. - Assigned during system development in the Platform Designer.
<destination>	Identifies the register that stores the result of the result port (if any).
<source 0>	Identifies the register that provides the first input argument from the data0 port (if any).
<source 1>	Identifies the register that provides the second input argument from the data1 port (if any).

The custom instruction designates the registers based on the application binary interface (ABI) name of each registers.

Example 3. Custom Instruction Assembly Language Syntax

```
CUSTOM_OPERATION1, a5, a5, a4
```

The example above shows a call to a custom instruction, called CUSTOM_OPERATION1. The input to the instruction is the current contents of the Nios V processor registers a5 and a4. Nios V processor register a5 stores the results .

3.3. Custom Instruction Word Format

The Nios V processor custom instructions are based on the R-type instruction format. The instruction word specifies the selector index, extension index, immediate, ALU switch and register usage. The ctrl port delivers the 32-bit custom instruction word to the custom logic blocks.

Besides the opcode (hardcoded in Platform Designer), rs2, rs1, and rd fields (allocated during software compilation), you can assign funct7 and funct3 fields to serve different purposes. You can either hardcode the funct7 and funct3 fields in Platform Designer or translate the fields as input arguments in the software C-macro.

Table 6. 32-bit Custom Instruction Word Format

32-bit Custom Instruction Bit Field															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
funct7[6:0]								rs2				rs1			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rs1	funct3[2:0]					rd				opcode					

Each of the bit field serves the following purposes:

Table 7. Bits Description

Bits	Function
funct7[6:4]	- Optional selector index - User-defined immediate
funct7[3:0]	- Extension index - User-defined immediate
rs2	Source register of data1
rs1	Source register of data0
funct3[2:0]	- Extension index - ALU switch - User-defined immediate
rd	Destination register for result
opcode	Required selector index

3.3.1. Selector and Extension Indexes

Both selector and extension indexes determine the custom instruction to execute. They are made up of the funct7, funct3 and opcode fields. When applying this feature, Intel recommends that you hardcode the indexes in the Platform Designer. The table below explains their differences.

The Nios V processor supports up to 32 custom logic blocks (4 opcodes with 3-bit funct7[6:4]). Each custom logic block can implement a single instruction, or multiple instructions.

In the case of a simple (non-extended) custom instruction, the Nios V processor can support up to 32 custom instructions. In extended custom instructions, the selector index is added with the extension index, results in a maximum supported custom instruction of 4096 instructions (4 opcodes with 10-bit funct7 and funct3).

Table 8. Selector and Extension Indexes

Index	Description	Corresponding bit fields	Required?	Usability	Maximum Custom Instructions	Maximum Custom Logic Blocks
Selector	Selects custom logic block that the instruction resides	opcode	Yes	In every custom instruction	4 (1 per custom logic blocks)	4
		funct7[6:4]	No	Only when assigning more than four custom logic blocks.	32 (1 per custom logic blocks)	32
Extension	Selects custom instruction in a single custom logic block	- funct7[3:0] - funct3[2:0]	No	Only in extended custom instructions	4096 (128 per custom logic blocks)	32 <i>Note:</i> Depends on the selector index.

Figure 9. Selector Index – opcode and funct7[6:4]

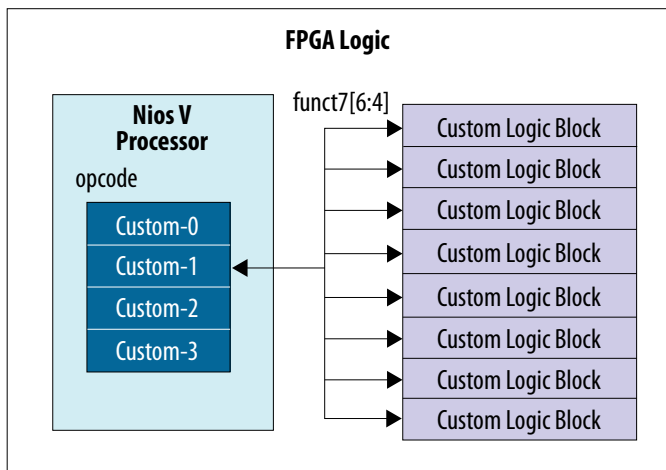
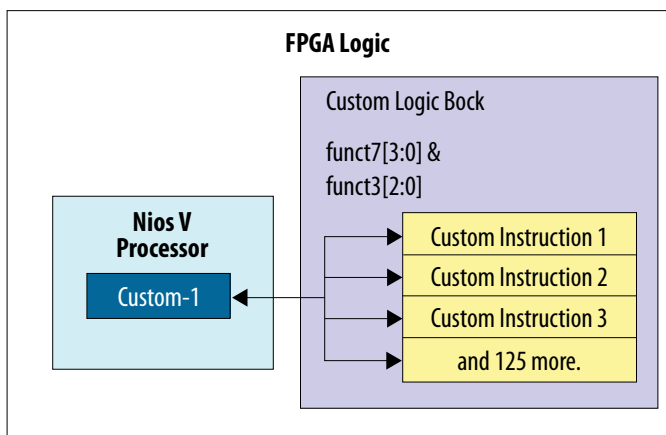


Figure 10. Extension Index - funct7[3:0] and funct3[2:0]



3.3.2. ALU Switch

The funct3 field determines the ALU switch. Using data0 and data1 as inputs, the ALU switch picks an ALU operation, and delivers the result as alu_result at the start of the custom instruction. You can use the alu_result, concurrently with the same data0 and data1 in the custom instruction. When applying this feature, Intel recommends you to hardcode the funct3 value in the Platform Designer.

Table 9. ALU Switch Options

funct3	ALU Operation	alu_result[31:0]
000	ADD	Addition of data0 and data1.
001	Reserved	N/A
010	SLT (Set on Less Than (signed))	1 if data0 < data1, 0 otherwise.
continued...		

funct3	ALU Operation	alu_result[31:0]
011	SLTU (Set on Less Than (unsigned))	1 if data0 < data1, 0 otherwise.
100	XOR (Exclusive OR)	Bitwise XOR operation of data0 and data1.
101	Reserved	N/A
110	OR	Bitwise OR operation of data0 and data1.
111	AND	Bitwise AND operation of data0 and data1.

Figure 11. ALU Switch Block Diagram

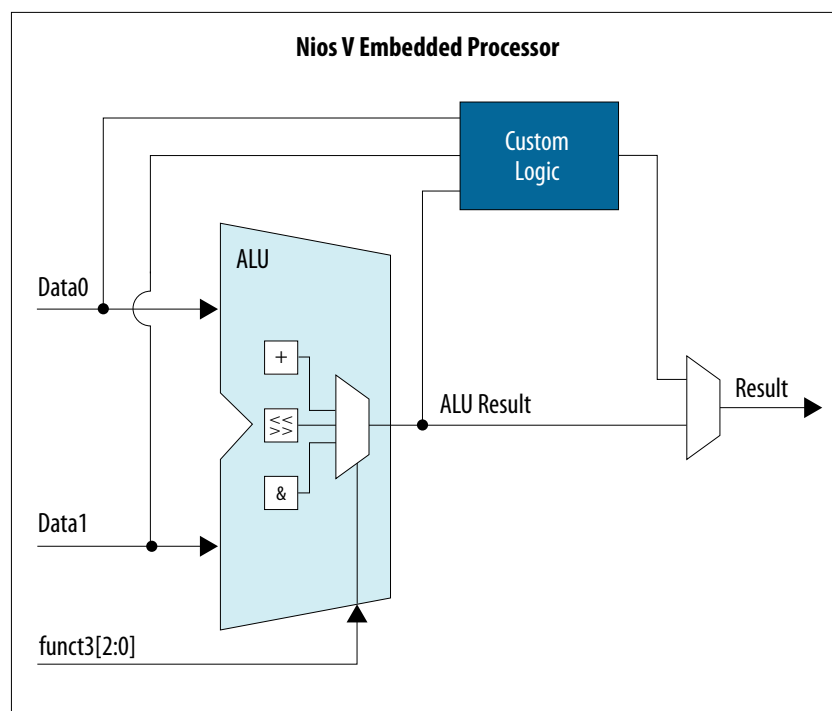
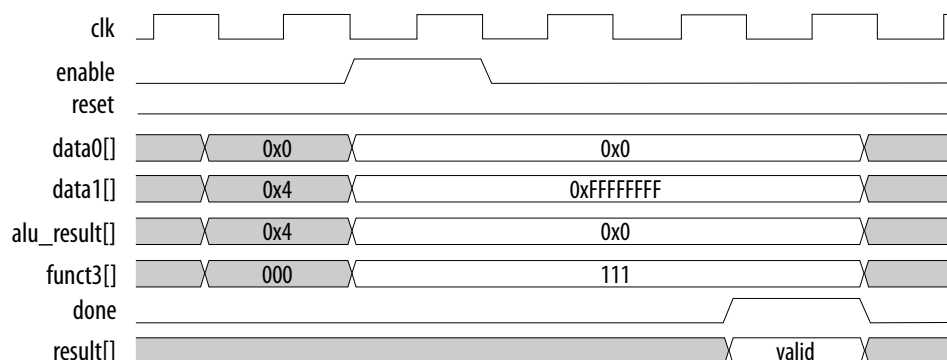


Figure 12. ALU Switch Timing Diagram with AND Operation



3.3.3. User-defined Immediate

You can apply the funct7 or funct3 field as a user-defined immediate by assigning "x" in the Software C-Macro, in the Platform Designer. Assigning the logic translates the fields into input arguments in the Software C-Macro.

Note: Platform Designer translates the bit field to the user-defined immediate when assigning the logic "x" in Software C-Macro only.

Example 4. Software C Macros with funct3 and funct7 fields assigned with "x"

```
#define CUSTOM_OPERATION(VAL_1, VAL_2, F3, F7) ({ \
    int output; \
    asm volatile (".insn r 0x2B, %[FUNCT3], (0x0 | (%[FUNCT7]<<0)), %[out], %\n" \
        : [out] "=r" (output) \
        : [input1] "r" (VAL_1), [input2] "r" (VAL_2), [FUNCT3] "i" (F3), [FUNCT7] \
        "i" (F7)); \
    output; \
})
```

4. Designing with Intel Quartus Prime and Platform Designer

4.1. Interface Signals

Table 10. Interface Signals

Feature	Signal	Width	Direction	Description
Custom Instruction Interface	clk	1	Manager to Subordinate	System clock
	reset	1	Manager to Subordinate	Synchronous reset
	data0	32	Manager to Subordinate	Input operand to custom instruction logic block (Source Register 1)
	data1	32	Manager to Subordinate	Input operand to custom instruction logic block (Source Register 2)
	alu_result	32	Manager to Subordinate	Input operand to custom instruction logic block (ALU switch result)
	ctrl	32	Manager to Subordinate	Custom instruction word
	enable	1	Manager to Subordinate	Custom instruction logic block enable
	done	1	Subordinate to Manager	Indicator to the completion of custom instruction execution
	result	32	Subordinate to Manager	Result of custom instruction

4.1.1. alu_result port

The `alu_result` port delivers 32-bit ALU outputs at the start of the custom instruction according to the ALU switch (`funct3[2:0]`). Refer to the *ALU Switch* for the available ALU operations.

Related Information

[ALU Switch](#) on page 16

4.1.2. ctrl port

The `ctrl` port presents the 32-bit custom instruction from the Nios V processor to the custom logic block. Refer to the *Custom Instruction Word Format* for the function of each field.

Related Information

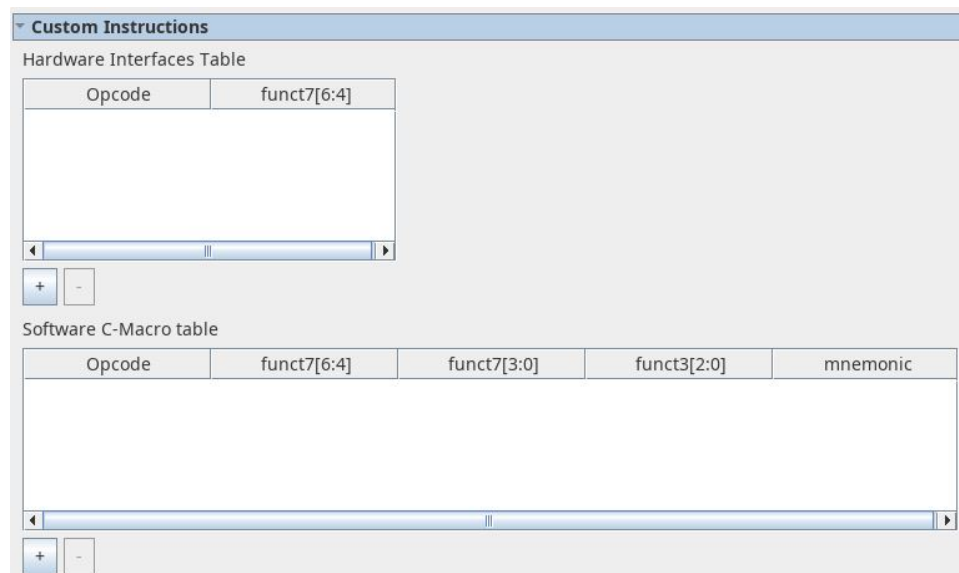
[Custom Instruction Word Format](#) on page 14

4.2. Custom Instruction Manager

After instantiating the Nios V processor IP core, you can instantiate multiple custom instruction interfaces (up to 32) from the processor core. In addition, each custom instruction interface can implement extended custom instruction to support up to 4096 custom instructions.

Configure the custom instruction in **Hardware Interfaces Table** and **Software C-Macro Table** to suit your needs.

Figure 13. Custom Instruction Tab



4.2.1. Hardware Interfaces Table

You must assign the opcode and funct7 field of a custom instruction logic block. Select the symbol + to add new assignment, and - to delete. Each assignment results in a single Nios V Custom Instruction Manager interface at the processor core.

Table 11. Hardware Interfaces Table Parameter

Parameter	Description
Opcode	- The required selector index. - String assignment, with CUSTOM0, CUSTOM1, CUSTOM2, or CUSTOM3 as options.
funct7[6:4]	- As optional selector index, or user-defined immediate. 3-bits assignment, with available logic of 0, 1, or x. - Assign the bits as x to register the bit for user-defined immediate.

Figure 14. Example Assignments in Hardware Interface Table

Opcode	funct7[6:4]
CUSTOM0	000
CUSTOM0	111
CUSTOM1	xxx
CUSTOM2	0xx
CUSTOM3	10x

Figure 15. Nios V Custom Instruction Manager Examples

cpu	Nios V/g Processor Intel FPGA IP
clk	Clock Input
reset	Reset Input
platform_irq_rx	Interrupt Receiver
instruction_manager	AXI4 Manager
data_manager	AXI4 Manager
ci_custom0_000	Nios V Custom Instruction Manager
ci_custom0_111	Nios V Custom Instruction Manager
ci_custom1	Nios V Custom Instruction Manager
ci_custom2_0xx	Nios V Custom Instruction Manager
ci_custom3_10x	Nios V Custom Instruction Manager
timer_sw_agent	Avalon Memory Mapped Agent
dm_agent	Avalon Memory Mapped Agent

4.2.2. Software C-Macro Table

Unlike the **Hardware Interfaces Table**, the **Software C-Macro Table** configures the custom instruction software C Macro for software development. Note that, any assignment made need to correlates to the earlier assignments in **Hardware Interfaces Table**.

Select the symbol + to add new assignment, and - to delete it. Each assignment results in a single Nios V processor custom instruction Software C Macro in the system.h file.

Table 12. Software C-Macro Table Parameter

Parameter	Description
Opcode	- Correlate to the Hardware Interfaces Table. - The required selector index. - String assignment, with CUSTOM0, CUSTOM1, , CUSTOM2, or CUSTOM3 as options.
funct7[6:4]	- Correlate to the Hardware Interfaces Table. - As optional selector index, or user-defined immediate. 3-bits assignment, with available logic of 0, 1, or x. - Assign any bits as x to register it as input argument in the C Macro.
funct7[3:0]	- As extension index, or user-defined immediate. 4-bits assignment, with available logic of 0, 1, or x. - Assign any bits as x to register it as input argument in the C Macro.
funct3[2:0]	- As extension index, ALU switch, or user-defined immediate. 3-bits assignment, with available logic of 0, 1, or x. - Assign any bits as x to register it as input argument in the C Macro.
mnemonic	- Define the mnemonic name of the custom instruction. - Intel recommends you to capitalize the name.

Figure 16. Example Assignments in Software C-Macro Table

Software C-Macro table

Opcode	funct7[6:4]	funct7[3:0]	funct3[2:0]	mnemonic
CUSTOM0	000	0000	000	CUSTOM_0_000
CUSTOM0	111	0000	000	CUSTOM_0_111
CUSTOM1	xxx	xxxx	xxx	CUSTOM_1_XXX
CUSTOM2	001	xxxx	xxx	CUSTOM_2_0XX
CUSTOM3	101	0000	000	CUSTOM_3_10X

Example 5. C Macros in system.h

```
#define CUSTOM_0_000(VAL_1, VAL_2) ({ \
    int output; \
    asm volatile (".insn r 0x0B, 0x0, 0x0, %[out], %[input1], %[input2]" \
        : [out] "=r" (output) \
        : [input1] "r" (VAL_1), [input2] "r" (VAL_2)); \
    output; \
})

#define CUSTOM_0_111(VAL_1, VAL_2) ({ \
    int output; \
    asm volatile (".insn r 0x0B, 0x0, 0x70, %[out], %[input1], %[input2]" \
        : [out] "=r" (output) \
        : [input1] "r" (VAL_1), [input2] "r" (VAL_2)); \
    output; \
})

#define CUSTOM_1_XXX(VAL_1, VAL_2, F3, F7) ({ \
    int output; \
    asm volatile (".insn r 0x2B, %[FUNCT3], (0x0 | (%[FUNCT7]<<0)), %[out], % \
        : [out] "=r" (output) \
        : [input1] "r" (VAL_1), [input2] "r" (VAL_2), [FUNCT3] "i" (F3), [FUNCT7] \
        "i" (F7)); \
    output; \
})
```

```
#define CUSTOM_2_0XX(VAL_1, VAL_2, F3, F7) ({ \
    int output; \
    asm volatile (".insn r 0x5B, %[FUNCT3], (0x10 | (%[FUNCT7]<<0)), %[out], %\
    [input1], %[input2]" \
        : [out] "=r" (output) \
        : [input1] "r" (VAL_1), [input2] "r" (VAL_2), [FUNCT3] "i" (F3), [FUNCT7] \
        "i" (F7)); \
    output; \
})

#define CUSTOM_3_10X(VAL_1, VAL_2) ({ \
    int output; \
    asm volatile (".insn r 0x7B, 0x0, 0x50, %[out], %[input1], %[input2]" \
        : [out] "=r" (output) \
        : [input1] "r" (VAL_1), [input2] "r" (VAL_2)); \
    output; \
})
```

4.3. Custom Instruction Subordinate

You can use the **Platform Designer Component Editor** to instantiate a Nios V processor custom instruction subordinate based on your custom hardware. The Platform Designer component editor enables you to create new components in the Platform Designer system.

Implementing a custom instruction involves using the custom instruction tool flow and comprises of the following tasks:

1. Writing the custom instruction HDL file
2. Opening the component editor
3. Specifying the custom instruction component type
4. Displaying the custom instruction block symbol
5. Adding the HDL files
6. Configuring the custom instruction parameter type
7. Setting up the custom instruction interfaces
8. Saving and adding the custom instruction
9. Generating the system and compiling in the Intel Quartus Prime software

4.3.1. Writing the Custom Instruction HDL File

Instantiate the **Custom Instruction Manager**, then begin writing the custom instruction HDL file. You can start by using the following example:

```
module custom_instruction(
    input clk,
    input reset,
    input [31:0] data0,
    input [31:0] data1,
    input [31:0] ctrl,
    input [31:0] alu_result,
    input enable,
    output done,
    output [31:0] result
);

//begin the custom instruction here.

endmodule
```

4.3.2. Opening the Component Editor

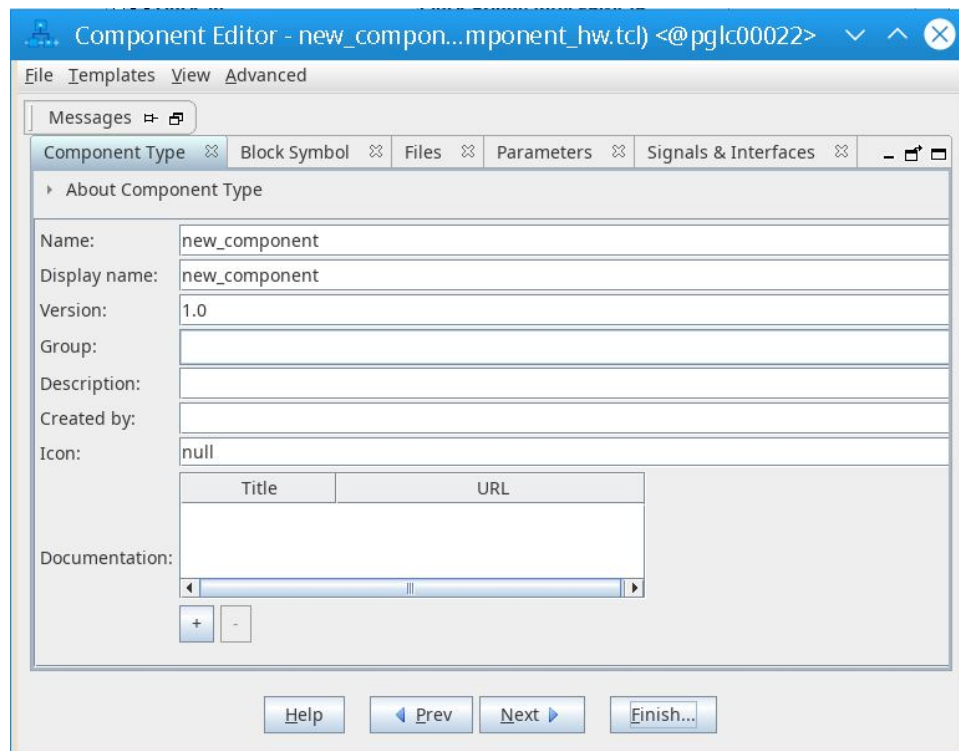
After you complete writing the custom instruction HDL file, you can open the Platform Designer and the Component Editor.

1. Navigate to **Tools**, click **Platform Designer**
2. Open the processor system QSYS file
3. Go to **File** ► **New Component...**
4. Open the **Component Editor**

4.3.3. Specifying the Custom Instruction Component Type

To specify the custom instruction component type, you can fill in a name, a display name, a version, and optionally a group, creator, and icon. Intel recommends you to add the description. These steps help define the `_hw.tcl` file for the new custom component.

Figure 17. Component Editor – Component Type

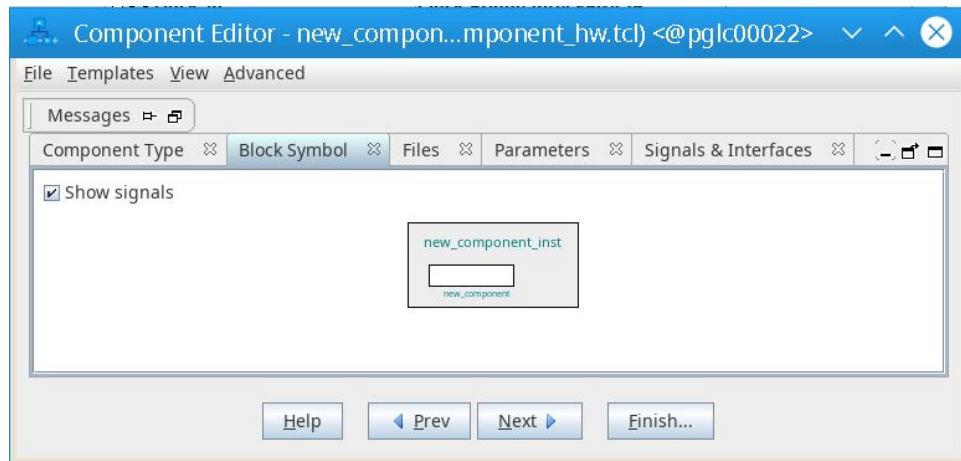


4.3.4. Displaying the Custom Instruction Block Symbol

Click **Next** to display the custom component in the **Block Symbol** tab.

Note: You need to add the HDL file to preview the final block symbol.

Figure 18. Component Editor – Block Symbol



4.3.5. Adding the Custom Instruction HDL File

To specify the synthesis HDL files for your custom instruction, browse to the HDL logic definition files in the design example.

To specify the synthesis files, follow these steps:

1. Click **Next** to display the **Files** tab.
2. Under **Synthesis Files**, click **Add Files**.
3. Browse to the custom instruction HDL file and click **Open**.

Note: The Intel Quartus Prime Analysis and Synthesis program checks the design for errors when you add the files. Ensure that no error message appears.

4. If there are multiple HDL files, follow the steps below. Else, proceed to the next step.
 - a. Go to the top-level HDL file.
 - b. At the same row, click the **Attributes** column.
 - c. Turn on **Top-level File** in the **File Attributes** window.
 - d. Click **OK**.
5. Click **Analyze HDL Files** to synthesize the top-level file.
6. To simulate the system, you can add your simulation files under **Verilog Simulation Files** or **VHDL Simulation Files** in the **Files** tab.

Figure 19. Component Editor – Synthesis Files

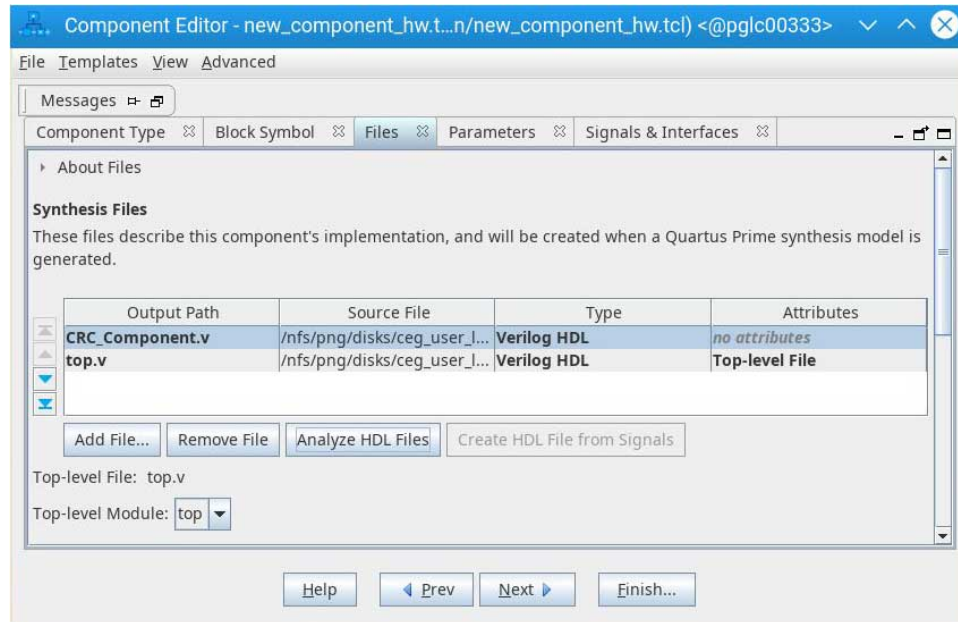


Figure 20. Component Editor – Verilog Simulation Files

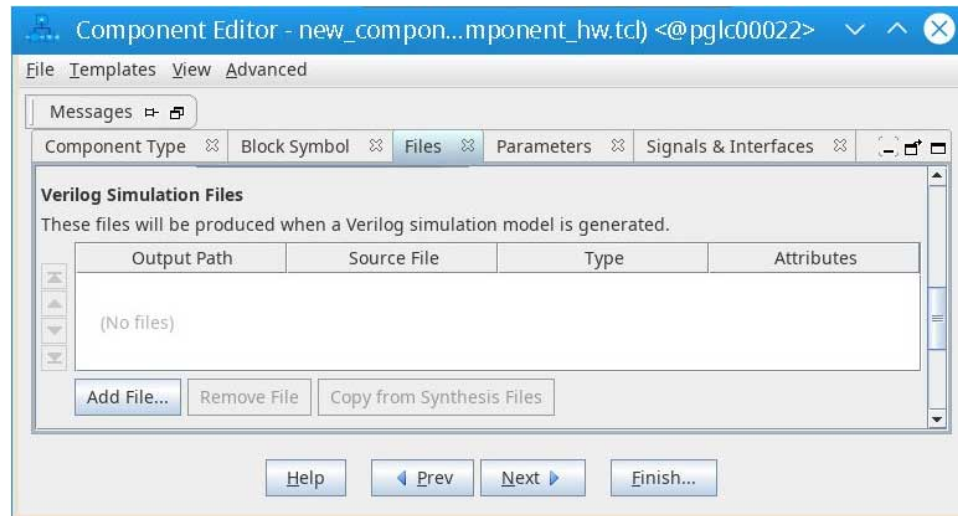
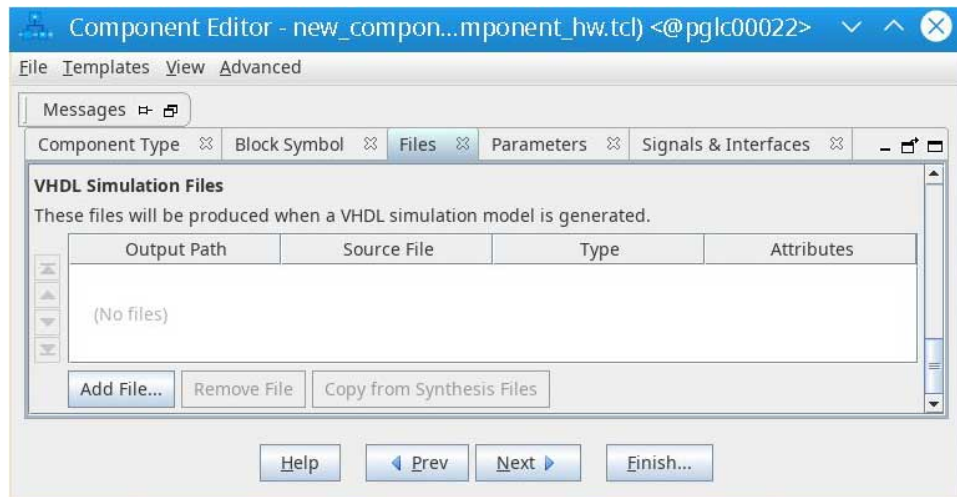


Figure 21. Component Editor – VHDL Simulation Files



4.3.6. Configuring the Custom Instruction Parameter Type

1. Click **Next** to display the **Parameters** tab.
2. To remove a parameter from the custom instruction parameter editor, you can uncheck the **Editable** box in the **Parameter** tab. The value defaults to the value in the **Default value** column.
 - When you uncheck the **Editable** box, you cannot see or change the parameter.
 - When you check the **Editable** box, you can see and change the parameter value.

To preview the custom component's parameter editor, click **Preview the GUI**.

Figure 22. Component Editor – Parameters Tab

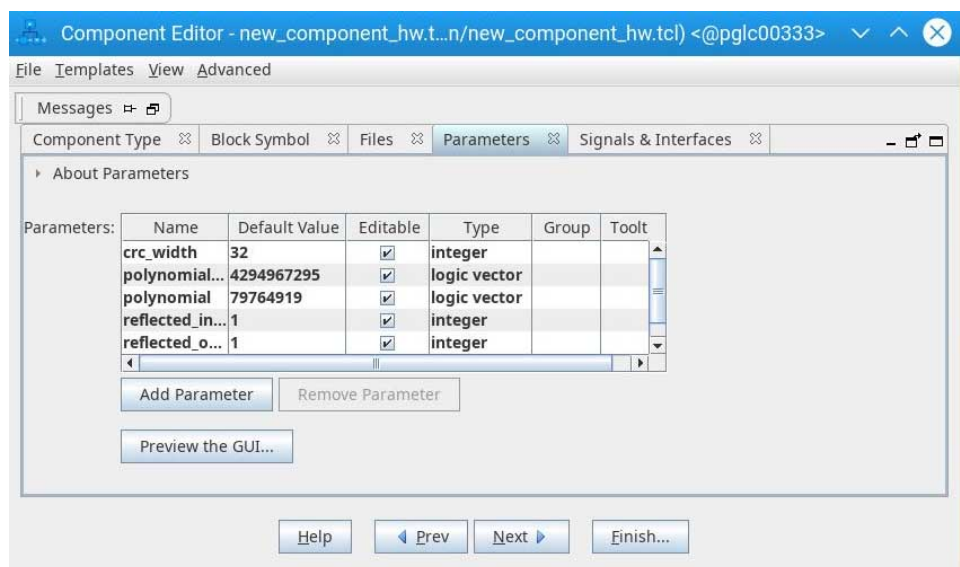


Figure 23. IP Parameter Editor – Parameters Tab

Parameters	
crc_width:	32
polynomial_init:	-1
polynomial:	79764919
reflected_input:	1
reflected_output:	1
xor_output:	-1
clock0:	0
clock1:	1

Related Information

[Intel Quartus Prime Pro Edition User Guide: Platform Designer](#)

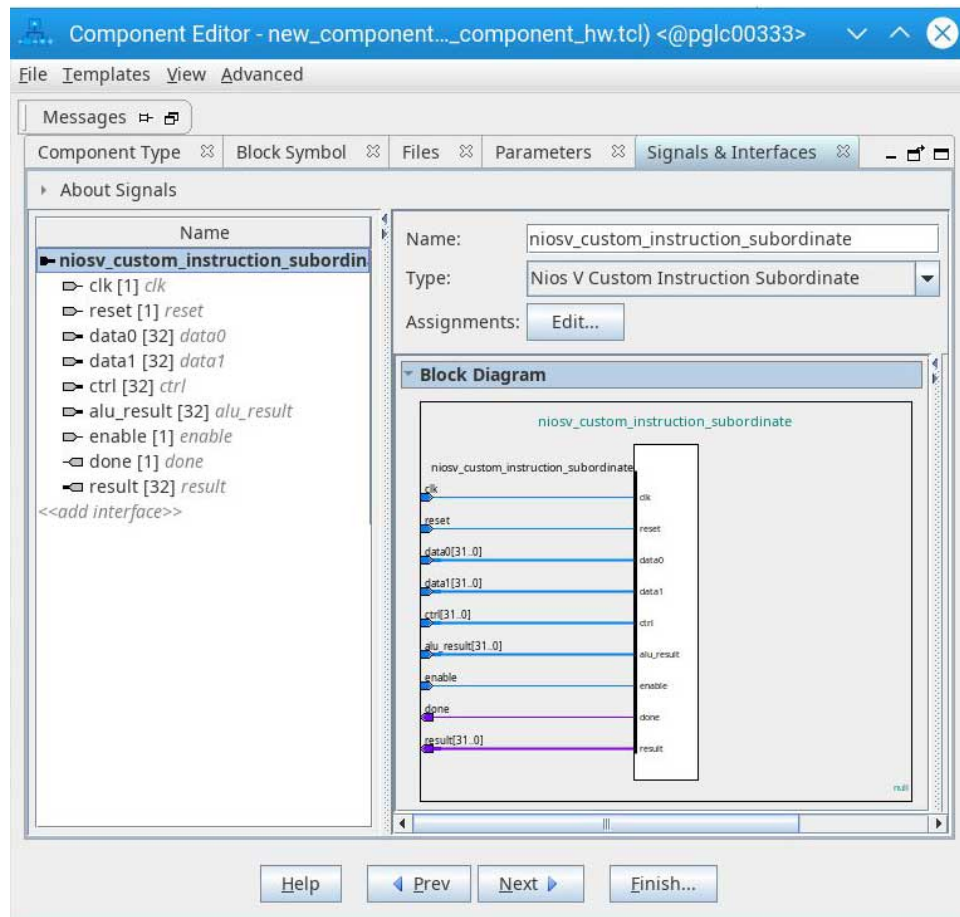
Refer to the topic [Parameter Type Properties](#) for more information about the parameter type properties.

4.3.7. Setting Up the Custom Instruction Interfaces

To set up the custom instruction interfaces, follow these steps:

1. Click **Next** to display the **Signals and Interfaces** tab. The **Signals and Interfaces** tab displays the interface of the synthesized HDL file.
2. Change all signals **Type** to **niosv_custom_instruction_subordinate**.
Note: You must include the clock input and reset input under the **Nios V Custom Instruction Subordinate**.
3. Update each HDL signal to the appropriate **Signal Type**.

Figure 24. Component Editor – Signals & Interfaces



4.3.8. Saving and Adding the Custom Instruction

To save and add the custom instruction to the Nios V processor, follow these steps:

1. Click **Finish**. A dialog box prompts you to save your changes before exiting.
2. Click **Yes, Save**. The new custom instruction appears in the Platform Designer Component Library.
3. In the Platform Designer Component Library, under Library, search for the newly created custom instruction component.
4. Click **Add** to add the new instruction to the Platform Designer system.
5. In the **Connections** panel, connect the **Nios V Custom Instruction Subordinate** interface to the **Nios V Custom Instruction Manager** interface.

Figure 25. Custom Instruction Connection



4.3.9. Generating and Compiling the Processor System

After you add the custom instruction logic to the system, you can generate and compile the system in the Intel Quartus Prime software.

To generate the system and compile, follow these steps:

1. In Platform Designer, click **Generate HDL**.
2. Specify output file generation options.
3. Click **Generate**.
4. After system generation completes, on the **File** menu, click **Exit**.
5. On the **Processing** menu, click **Start Compilation**.

5. Document Revision History for AN 977: Nios V Processor Custom Instruction

Document Version	Changes
2023.04.14	Initial release.